



**Politechnika  
Śląska**

## **PRACA MAGISTERSKA**

System przeznaczony dla osób niewidomych do rozpoznawania dotykanych  
obiektów 3D

**Piotr MARCOL**

**Nr albumu: 300463**

**Kierunek:** Informatyka

**Specjalność:** Oprogramowanie Systemowe

**PROWADZĄCY PRACĘ**

**Dr hab. inż., prof. PŚ Michał Maćkowski**

**KATEDRA Systemów Rozproszonych i Urządzeń Informatyki**

**Wydział Automatyki, Elektroniki i Informatyki**

**Gliwice 2026**



**Tytuł pracy**

System przeznaczony dla osób niewidomych do rozpoznawania dotykanych obiektów 3D

**Streszczenie**

(Streszczenie pracy – odpowiednie pole w systemie APD powinno zawierać kopię tego streszczenia.)

**Słowa kluczowe**

(2-5 słów (fraz) kluczowych, oddzielonych przecinkami)

**Thesis title**

A system designed for the blind to recognize touched 3D objects

**Abstract**

(Thesis abstract – to be copied into an appropriate field during an electronic submission – in English.)

**Key words**

(2-5 keywords, separated by commas)



# Spis treści

|          |                                                                       |          |
|----------|-----------------------------------------------------------------------|----------|
| <b>1</b> | <b>Wstęp</b>                                                          | <b>1</b> |
| 1.1      | Motywacja podjęcia tematu . . . . .                                   | 1        |
| 1.2      | Cel i zakres pracy . . . . .                                          | 1        |
| 1.3      | Główne założenia i ograniczenia . . . . .                             | 1        |
| 1.4      | Wkład autora . . . . .                                                | 2        |
| 1.5      | Struktura pracy . . . . .                                             | 2        |
| <b>2</b> | <b>Analiza problemu i przegląd literatury</b>                         | <b>3</b> |
| 2.1      | Technologie wspomagające osoby z niepełnosprawnością wzroku . . . . . | 3        |
| 2.2      | Problem rozpoznawania obiektów 3D w interakcji dotykowej . . . . .    | 3        |
| 2.3      | Metody detekcji obiektów w obrazach . . . . .                         | 3        |
| 2.4      | Rodzina modeli YOLO . . . . .                                         | 3        |
| 2.5      | Wykorzystanie danych RGB, głębi i LiDAR w rozpoznawaniu obiektów . .  | 4        |
| 2.6      | Metryki oceny detekcji obiektów . . . . .                             | 4        |
| 2.7      | Podsumowanie analizy tematu . . . . .                                 | 6        |
| <b>3</b> | <b>Koncepcja systemu rozpoznawania dotykanych obiektów 3D</b>         | <b>7</b> |
| 3.1      | Założenia funkcjonalne i нефункционалне . . . . .                     | 7        |
| 3.2      | Architektura systemu . . . . .                                        | 7        |
| 3.3      | Dobór klas rozpoznawanych obiektów . . . . .                          | 7        |
| 3.4      | Uzasadnienie wyboru metody detekcji . . . . .                         | 7        |
| 3.5      | Przepływ danych w systemie . . . . .                                  | 8        |
| 3.6      | Ograniczenia przyjętej koncepcji . . . . .                            | 8        |
| <b>4</b> | <b>Implementacja systemu</b>                                          | <b>9</b> |
| 4.1      | Środowisko programistyczne i wykorzystane narzędzia . . . . .         | 9        |
| 4.2      | Moduł akwizycji danych obrazowych . . . . .                           | 9        |
| 4.3      | Moduł komunikacji klient-serwer . . . . .                             | 9        |
| 4.4      | Moduł detekcji obiektów . . . . .                                     | 9        |
| 4.5      | Moduł informacji zwrotnej dla użytkownika . . . . .                   | 9        |
| 4.6      | Organizacja kodu i plików projektu . . . . .                          | 10       |

|          |                                                                               |           |
|----------|-------------------------------------------------------------------------------|-----------|
| <b>5</b> | <b>Zbiór danych i metodyka badań</b>                                          | <b>11</b> |
| 5.1      | Charakterystyka zbioru danych . . . . .                                       | 11        |
| 5.2      | Proces pozyskiwania i oznaczania danych . . . . .                             | 11        |
| 5.3      | Podział danych na zbiory treningowy, walidacyjny i testowy . . . . .          | 11        |
| 5.4      | Warianty danych wejściowych . . . . .                                         | 11        |
| 5.5      | Konfiguracja treningu modeli . . . . .                                        | 12        |
| 5.6      | Metryki oceny . . . . .                                                       | 12        |
| 5.7      | Procedura eksperymentalna . . . . .                                           | 12        |
| <b>6</b> | <b>Wyniki badań i ich analiza</b>                                             | <b>13</b> |
| 6.1      | Wyniki uczenia modeli YOLO . . . . .                                          | 13        |
| 6.2      | Porównanie wariantów modeli . . . . .                                         | 13        |
| 6.3      | Wpływ reprezentacji obrazu na skuteczność detekcji . . . . .                  | 13        |
| 6.4      | Analiza błędów klasyfikacji i detekcji . . . . .                              | 13        |
| 6.5      | Ocena działania systemu w scenariuszu użytkowym . . . . .                     | 14        |
| 6.6      | Dyskusja ograniczeń wyników . . . . .                                         | 14        |
| <b>7</b> | <b>Podsumowanie i wnioski końcowe</b>                                         | <b>15</b> |
| 7.1      | Podsumowanie wykonanych prac . . . . .                                        | 15        |
| 7.2      | Ocena realizacji celu pracy . . . . .                                         | 15        |
| 7.3      | Najważniejsze wnioski . . . . .                                               | 15        |
| 7.4      | Możliwe kierunki rozwoju . . . . .                                            | 15        |
|          | <b>Bibliografia</b>                                                           | <b>17</b> |
|          | <b>Dokumentacja techniczna</b>                                                | <b>21</b> |
|          | <b>Spis skrótów i symboli</b>                                                 | <b>23</b> |
|          | <b>Lista dodatkowych plików, uzupełniających tekst pracy (jeżeli dotyczy)</b> | <b>25</b> |
|          | <b>Spis rysunków</b>                                                          | <b>27</b> |
|          | <b>Spis tabel</b>                                                             | <b>29</b> |

# Rozdział 1

## Wstęp

To powinien być rozdział „ustawiający” całą pracę. Powinny się tu znaleźć:

- kontekst: technologie wspomagające osoby z niepełnosprawnością wzroku,
- problem: rozpoznawanie dotykanych obiektów 3D w czasie zbliżonym do rzeczywistego,
- cel pracy,
- zakres pracy,
- wkład własny,
- krótka charakterystyka rozdziałów.

### 1.1 Motywacja podjęcia tematu

Tu piszę, dlaczego temat ma sens: samodzielność osób niewidomych, rozpoznawanie obiektów, ograniczenia obecnych rozwiązań.

### 1.2 Cel i zakres pracy

Cel np.: „Celem pracy jest opracowanie i eksperymentalna ocena systemu wspomagającego rozpoznawanie dotykanych obiektów 3D z wykorzystaniem metod detekcji obiektów na obrazie.”

### 1.3 Główne założenia i ograniczenia

Tu można wpisać: iPhone jako urządzenie wejściowe, obiekty drukowane 3D, sześć klas, detekcja w obrazie, komunikacja z serwerem, informacja zwrotna audio.

## 1.4 Wkład autora

Bardzo ważne, bo wymagania tego chcą. Wkład: przygotowanie datasetu, implementacja klienta/serwera, trening modeli, eksperymenty, analiza wyników.

## 1.5 Struktura pracy

Krótki opis, co jest w każdym rozdziale.



## Rozdział 2

# Analiza problemu i przegląd literatury

To jest główny wstęp teoretyczny / state of the art. Wymagania mówią wprost, że analiza tematu ma zawierać wprowadzenie do dziedziny, sformułowanie problemu, poszerzone studia literaturowe, przegląd rozwiązań i algorytmów.

### 2.1 Technologie wspomagające osoby z niepełnosprawnością wzroku

Aplikacje mobilne, systemy wearable, kamery, czujniki głębi, audio feedback.

### 2.2 Problem rozpoznawania obiektów 3D w interakcji dotykowej

Tutaj opisuję, że użytkownik nie tylko patrzy kamerą na scenę, ale dotyka obiektu, więc problem jest trochę inny niż klasyczna detekcja przedmiotów w pomieszczeniu.

### 2.3 Metody detekcji obiektów w obrazach

YOLO, SSD, Faster R-CNN, Mask R-CNN — ogólnie, bez przesadnego grzebania.

### 2.4 Rodzina modeli YOLO

Tu najwięcej, bo tego używasz. Architektura ogólnie, zalety: szybkość, detekcja w czasie rzeczywistym, różne rozmiary modeli.

## 2.5 Wykorzystanie danych RGB, głębi i LiDAR w rozpoznawaniu obiektów

Tu mogę opisać, dlaczego LiDAR był rozważany, ale w praktyce ma ograniczenia przy małych obiektach i małych odległościach.

## 2.6 Metryki oceny detekcji obiektów

Ocena modeli detekcji obiektów wymaga zastosowania odpowiednich metryk, które pozwalają na porównanie uzyskanych wyników. Metryki te powinny uwzględniać zarówno poprawność klasyfikacji, jak i dokładność lokalizacji obiektu na obrazie. Przeciwnie do klasyfikacji obrazów, przy detekcji nie wystarczy, aby model poprawnie rozpoznał klasę obiektu. Równie istotne jest, aby poprawnie wyznaczył ramkę ograniczającą wokół wykrytego obiektu. Jedną z najczęściej stosowanych metryk jest IoU (ang. *Intersection over Union*), określająca stopień pokrycia między ramką obiektu przewidywaną przez model a ramką rzeczywistą [3]. Wartość IoU obliczana jest jako stosunek pola części wspólnej obu ramek do pola ich części łącznej:

$$IoU = \frac{|B_p \cap B_{gt}|}{|B_p \cup B_{gt}|}, \quad (2.1)$$

gdzie  $B_p$  jest ramką przewidywaną przez model, a  $B_{gt}$  jest ramką referencyjną (ang. *ground truth*). Im większa wartość IoU, tym lepsze dopasowanie ramki przewidywanej do ramki rzeczywistej. W praktyce najczęściej stosuje się progi, np.  $IoU \geq 0.5$ , aby zdecydować, czy dana detekcja może być uznana za poprawną. Próg ten został wykorzystany w wielu klasycznych procedurach oceny detekcji, takich jak PASCAL VOC [1], i jest nadal powszechnie stosowany w wielu benchmarkach. Na podstawie wartości IoU oraz zgodności klasy obiektu możliwe jest przypisanie wyników detekcji do jednej z kategorii: prawdziwych pozytywów (TP, ang. *true positive*), fałszywych pozytywów (FP, ang. *false positive*) i fałszywych negatywów (FN, ang. *false negative*). Detekcja jest traktowana jako TP, jeżeli model poprawnie rozpoznał klasę obiektu, a przewidywana ramka spełnia zadany próg IoU względem ramki referencyjnej. FP to detekcja błędna w której model przewidział klasę obiektu, ale nie spełnił progu IoU lub przewidziana klasa była błędna. FN natomiast to sytuacja, w której model nie wykrył obiektu, który rzeczywiście był na obrazie.

Jedną z podstawowych metryk oceny jest precyzja (ang. *precision*), określająca udział poprawnych detekcji modelu wśród wszystkich detekcji, jakie zwrócił:

$$Precision = \frac{TP}{TP + FP}. \quad (2.2)$$

Wysoka precyzja oznacza, że model generuje niewiele fałszywych pozytywów. Drugą,

powiązaną z nią metryką, jest czułość (ang. *recall*), określająca, jaki jest udział poprawnych detekcji wśród wszystkich rzeczywistych obiektów:

$$Recall = \frac{TP}{TP + FN}. \quad (2.3)$$

Obie metryki pozostają ze sobą w relacji kompromisu, ponieważ zwiększenie progu pewności detekcji może ograniczyć liczbę fałszywych pozytywów, ale jednocześnie prowadzić do pominięcia części rzeczywistych obiektów. Z kolei obniżenie tego progu może zwiększyć liczbę detekcji, ale równocześnie powodować wzrost liczby fałszywych detekcji. Z tego względu, przy analizie detektorów często łączy się je w jedną metrykę – krzywą PR (precyzja-czułość, ang. *precision-recall curve*), przedstawiającą zależność pomiędzy nimi dla różnych wartości progu decyzyjnego. Na jej podstawie wyznacza się metrykę AP (ang. *Average Precision*), odpowiadającą polu pod krzywą PR.

W przypadku detekcji wieloklasowej używa się średniej wartości AP obliczonej dla wszystkich klas, określanej jako mAP (ang. *mean Average Precision*):

$$mAP = \frac{1}{N} \sum_{i=1}^N AP_i, \quad (2.4)$$

gdzie  $N$  jest liczbą klas, a  $AP_i$  jest wartością AP dla  $i$ -tej klasy. W praktyce najczęściej wykorzystuje się dwie wersje tej metryki:  $mAP50$ , gdzie detekcję uznaje się za poprawną przy progu  $IoU \geq 0.5$ , oraz  $mAP50 - 95$ , gdzie wynik jest uśredniany dla progów  $IoU$  z zakresu od 0.5 do 0.95. Pierwsza z nich jest bardziej liberalna i pozwala określić ogólną zdolność modelu do wykrycia obiektu. Druga natomiast jest bardziej rygorystyczna, ponieważ wymaga dokładniejszego dopasowania ramek ograniczających, dzięki czemu lepiej pokazuje zarówno skuteczność klasyfikacji, jak i lokalizacji obiektu. Uśrednianie wyników dla wielu progów zostało szeroko zastosowane m. in. w metodyce oceny detekcji zbioru Microsoft COCO [2] i jest obecnie standardem w wielu benchmarkach detekcji obiektów.

Dodatkowo, w pracy wykorzystano metrykę  $F1$ -score, będącą średnią harmoniczną precyzji i czułości:

$$F1 = 2 \cdot \frac{P \cdot R}{P + R}, \quad (2.5)$$

gdzie  $P$  jest precyzją, a  $R$  jest czułością. Metryka ta umożliwia przedstawienie kompromisu pomiędzy liczbą fałszywych detekcji a liczbą pominiętych obiektów. Może być szczególnie przydatna, gdy sama precyzja lub czułość nie dają pełnego obrazu jakości modelu. W niniejszej pracy metrykę  $F1$ -score wykorzystano jako metrykę pomocniczą względem podstawowych metryk  $mAP50$  i  $mAP50 - 95$ , aby lepiej zrozumieć kompromis pomiędzy precyzją a czułością w kontekście detekcji obiektów dotykanych przez użytkownika.

Oprócz wartości liczbowych wykorzystano również macierze pomyłek (ang. *confusion*

*matrix*), pozwalające przeanalizować, które klasy obiektów są najczęściej mylone przez model. Są one przedstawiane w formie tabel, w których jedna oś odpowiada rzeczywistym klasom, a druga klasom przewidywanym przez model. Dla dobrze działającego modelu większość wartości powinna znajdować się na przekątnej macierzy, co wskazuje na poprawną klasyfikację obiektów. Macierze pomyłek są szczególnie istotne przy rozpoznawaniu brył geometrycznych o podobnych kształtach, np. sześcianu i prostopadłościanu, ponieważ sama wartość mAP nie wskazuje bezpośrednio, pomiędzy którymi klasami model popełnia najwięcej błędów. Analiza macierzy pomyłek może również pomóc w identyfikacji problemów z danymi treningowymi, np. niedostatecznej liczby przykładów dla wybranych klas lub błędów w oznaczeniach.

## 2.7 Podsumowanie analizy tematu

Najważniejsze: z literatury wynika, że detekcja obiektów w czasie rzeczywistym jest sensownym kierunkiem, ale Twoja nisza to rozpoznawanie konkretnych dotykanych brył 3D w kontrolowanym scenariuszu.

## Rozdział 3

# Koncepcja systemu rozpoznawania dotykanych obiektów 3D

To odpowiada wymaganej części „przedmiot pracy”: rozwiązanie zaproponowane przez dyplomanta, analiza teoretyczna, uzasadnienie metod, algorytmów i narzędzi.

### 3.1 Założenia funkcjonalne i нефunkcjonalne

Funkcjonalne: akwizycja obrazu, przesyłanie klatek, detekcja obiektu, informacja zwrotna. Niefunkcjonalne: możliwie niskie opóźnienie, prostota użycia, mobilność, powtarzalność badań.

### 3.2 Architektura systemu

iPhone → przesyłanie obrazu/WebSocket → serwer Python → model YOLO → wynik → feedback.

### 3.3 Dobór klas rozpoznawanych obiektów

Cube, sphere, cylinder, cuboid, tetrahedron, cone. Tu warto uzasadnić, że są to podstawowe bryły geometryczne, możliwe do wydruku 3D i łatwe do kontrolowanego oznaczania.

### 3.4 Uzasadnienie wyboru metody detekcji

Dlaczego YOLO: szybkość, popularność, gotowe narzędzia treningowe, możliwość porównania wariantów n/s, metryki.

## 3.5 Przepływ danych w systemie

Od obrazu z telefonu, przez detekcję, po komunikat dla użytkownika.

## 3.6 Ograniczenia przyjętej koncepcji

Na przykład: kontrolowany zestaw obiektów, zależność od oświetlenia, ograniczony dataset, problem z generalizacją na inne dłonie/osoby/tła.

# Rozdział 4

## Implementacja systemu

Tu opisuję konkretną robotę inżynierską. Nie za dużo kodu w głównym tekście — raczej architektura, moduły, decyzje. Opis interfejsów aplikacji i środowisk projektowania lepiej dawać jako załącznik, bo wymagania sugerują, że takie rzeczy powinny iść do załącznika.

### 4.1 Środowisko programistyczne i wykorzystane narzędzia

Python, PyTorch/Ultralytics, OpenCV, Swift/iOS, ARKit, WebSocket, CVAT/Robo-flow/HF — tylko te, które finalnie zostaną.

### 4.2 Moduł akwizycji danych obrazowych

Jak pozyskuję obraz z telefonu / nagrań / streamu.

### 4.3 Moduł komunikacji klient-serwer

WebSocket, przesyłanie klatek, format danych, obsługa połączenia.

### 4.4 Moduł detekcji obiektów

Ładowanie modelu, inferencja, wynik detekcji, progi confidence.

### 4.5 Moduł informacji zwrotnej dla użytkownika

Komunikaty audio / tekstowe / docelowe zachowanie aplikacji.

## 4.6 Organizacja kodu i plików projektu

Krótko, bez robienia instrukcji obsługi. Dokładna dokumentacja może iść do załącznika albo plików dodatkowych.



# Rozdział 5

## Zbiór danych i metodyka badań

To jest bardzo ważny rozdział, bo wymagania kładą nacisk na metodykę, dane, powtarzalność i naukową analizę wyników. Badania powinny być przeprowadzone „tak jak jest przyjęte w ujęciu naukowym”, np. z zapewnieniem powtarzalności testów.

### 5.1 Charakterystyka zbioru danych

Liczba klas, liczba obrazów, rozdzielczość, źródło danych, przykładowe obrazy.

### 5.2 Proces pozyskiwania i oznaczania danych

Nagrania, ekstrakcja klatek, anotacje, negatywne próbki, puste etykiety.

### 5.3 Podział danych na zbiory treningowy, walidacyjny i testowy

Train/val/test. Tu trzeba bardzo jasno wyjaśnić, jak dzielić dane, żeby recenzent nie zarzucił przecieku danych.

### 5.4 Warianty danych wejściowych

Kolor, grayscale, ewentualne filtry/normalizacja. To będzie dobre miejsce na porównanie „kolor vs grayscale”.

## 5.5 Konfiguracja treningu modeli

Modele, rozmiar obrazu, batch, liczba epok, patience, pretrained/from scratch, sprzęt GPU.

## 5.6 Metryki oceny

Precision, recall, mAP50, mAP50-95, macierz pomyłek, czas inferencji/FPS, zużycie pamięci.

Do oceny modeli wykorzystano metryki zwracane przez narzędzie Ultralytics YOLO: precision, recall, mAP50 oraz mAP50-95. Dodatkowo na podstawie wartości precision i recall obliczono F1-score. Wyniki zapisywano w pliku results.csv, natomiast przebiegi metryk oraz macierze pomyłek analizowano na podstawie plików generowanych po zakończeniu treningu i walidacji. [4]

## 5.7 Procedura eksperymentalna

Czyli dokładnie: jakie modele trenowano, na jakich danych, jakie wyniki zbierano, jak porównywano.

# Rozdział 6

## Wyniki badań i ich analiza

To powinien być osobny rozdział, nie mieszany z metodyką. Wymagania mówią, że badania mają zawierać prezentację wyników, opracowanie i poszerzoną dyskusję wyników oraz wnioski.

### 6.1 Wyniki uczenia modeli YOLO

Tabele z najlepszymi wynikami: model, imgsz, precision, recall, mAP50, mAP50-95, fl, czas, pamięć.

### 6.2 Porównanie wariantów modeli

YOLO n vs s, ewentualnie inne warianty.

### 6.3 Wpływ reprezentacji obrazu na skuteczność detekcji

Kolor vs grayscale. To może być jeden z ciekawszych fragmentów pracy.

### 6.4 Analiza błędów klasyfikacji i detekcji

Macierze pomyłek, najczęściej mylone klasy, np. cube/sphere/cuboid itd.

## 6.5 Ocena działania systemu w scenariuszu użytkowym

Nie tylko mAP, ale też: czy system nadaje się do użycia, jakie ma opóźnienia, co działa, co nie działa.

## 6.6 Dyskusja ograniczeń wyników

Mały/średni dataset, kontrolowane warunki, jedna osoba w części danych, zależność od tła/oświetlenia, możliwe przeuczenie, generalizacja.

# Rozdział 7

## Podsumowanie i wnioski końcowe

### 7.1 Podsumowanie wykonanych prac

Co zostało zrobione: dataset, modele, system, eksperymenty.

### 7.2 Ocena realizacji celu pracy

Wprost: czy cel został osiągnięty, w jakim zakresie, z jakimi ograniczeniami.

### 7.3 Najważniejsze wnioski

Np. detekcja obiektów jest skuteczna, ale ma ograniczenia, np. w przypadku małych obiektów, podobnych kształtów, zależności od tła i oświetlenia.

### 7.4 Możliwe kierunki rozwoju

Większy dataset, więcej osób, więcej obiektów, integracja LiDAR/RGB-D, inference lokalnie na iPhone, lepszy feedback audio, testy z użytkownikami.



# Bibliografia

- [1] Mark Everingham, Luc Van Gool, Christopher K. I. Williams, John Winn i Andrew Zisserman. „The PASCAL Visual Object Classes (VOC) Challenge.” W: *International Journal of Computer Vision* 88.2 (2010), s. 303–338. DOI: 10.1007/s11263-009-0275-4.
- [2] Tsung-Yi Lin, Michael Maire, Serge J. Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár i C. Lawrence Zitnick. „Microsoft COCO: Common Objects in Context”. W: *Computer Vision – ECCV 2014*. T. 8693. Lecture Notes in Computer Science. Springer, 2014, s. 740–755. DOI: 10.1007/978-3-319-10602-1\_48.
- [3] Rafael Padilla, Wesley L. Passos, Thadeu L. B. Dias, Sergio L. Netto i Eduardo A. B. da Silva. „A Comparative Analysis of Object Detection Metrics with a Companion Open-Source Toolkit”. W: *Electronics* 10.3 (2021). ISSN: 2079-9292. DOI: 10.3390/electronics10030279.
- [4] Ultralytics. *Performance Metrics Deep Dive*. YOLO Performance Metrics documentation. 12 list. 2023. URL: <https://docs.ultralytics.com/guides/yolo-performance-metrics/> (term. wiz. 17.12.2025).





# Dodatki



# Dokumentacja techniczna



# Spis skrótów i symboli

IoU Intersection over Union – miara stopnia pokrycia ramki przewidywanej przez model z ramką referencyjną.

TP True Positive – prawdziwy pozytywny, poprawnie wykryty obiekt.

FP False Positive – fałszywy pozytywny, błędna detekcja zwrócona przez model.

FN False Negative – fałszywy negatywny, obiekt rzeczywisty niewykryty przez model.

PR Precision-Recall – zależność między precyzją a czułością

AP Average Precision – średnia precyzja wyznaczana na podstawie krzywej precision-recall.

mAP mean Average Precision – średnia wartość AP obliczana dla wszystkich klas.

F1 F1-score – średnia harmoniczna precyzji i czułości.

VOC Visual Object Classes – zbiór nazw benchmarku PASCAL VOC wykorzystywanego w ocenie detekcji obiektów.

COCO Common Objects in Context – nazwa zbioru i benchmarku Microsoft COCO.



# Lista dodatkowych plików, uzupełniających tekst pracy (jeżeli dotyczy)

W systemie do pracy dołączono dodatkowe pliki zawierające:

- źródła programu,
- zbiory danych użyte w eksperymentach,
- film pokazujący działanie opracowanego oprogramowania lub zaprojektowanego i wykonanego urządzenia,
- itp.





# Spis rysunków



## Spis tabel